Báo cáo bằng chứng source code - Pick2Light (P2L)

Tài liệu này tổng hợp các đoạn source code quan trọng để chứng minh năng lực thực tế của hệ thống P2L. Mục tiêu là giúp quản lý hoặc người duyệt dự án có thể nhìn thấy bằng chứng kỹ thuật trực tiếp từ mã nguồn, không chỉ từ mô tả.

---

1. Kết luận nhanh

Từ source code hiện tại có thể xác nhận:

1. Hệ thống có lớp tích hợp MQTT hoàn chỉnh, gồm topic mặc định, subscribe command topic và Last Will status.
2. Hệ thống xử lý được nhiều loại lệnh nghiệp vụ: bật đèn, tắt đèn, batch picking, CSV task, scan thiết bị.
3. Business logic đã có cơ chế theo dõi batch và tự xác nhận hoàn tất khi toàn bộ module đã confirm.
4. Lớp giao tiếp phần cứng có cơ chế ACK ngay lập tức cho interrupt từ thiết bị AIOI và ghép response theo sequence number.
5. Web client có cơ chế giữ session sau khi reload trang và chỉ ngắt MQTT sau 15 phút idle.
6. Chức năng Scan Devices đã được triển khai end-to-end từ API backend đến giao diện frontend.

---

2. Bằng chứng 1 - MQTT topic mặc định đã được cấu hình trong chương trình

Ý nghĩa: chứng minh hệ thống đã có cấu hình MQTT thực tế cho command, event, device state và status.

Nguồn: `src/P2L.CLI/Program.cs:50-59`

```
Host              = Cfg_String(config, "Mqtt:Host",              "localhost"),
Port              = Cfg_Int   (config, "Mqtt:Port",              8883),
ClientId          = Cfg_String(config, "Mqtt:ClientId",          "p2l-bridge"),
Username          = config["Mqtt:Username"],
Password          = config["Mqtt:Password"],
CommandTopic      = Cfg_String(config, "Mqtt:CommandTopic",      "p2l/commands"),
EventTopic        = Cfg_String(config, "Mqtt:EventTopic",        "p2l/events"),
DevicesTopic      = Cfg_String(config, "Mqtt:DevicesTopic",      "p2l/devices"),
StatusTopic       = Cfg_String(config, "Mqtt:StatusTopic",       "p2l/status"),
OfflineBufferPath = Cfg_String(config, "Mqtt:OfflineBufferPath", Path.Combine(appDataBase, "
```

Nhận xét: Đây là bằng chứng rõ ràng cho thấy hệ thống không chỉ "hỗ trợ MQTT" ở mức ý tưởng, mà đã có cấu hình topic và cơ chế buffer offline trong mã nguồn chạy thật.

---

3. Bằng chứng 2 - MQTT Bridge subscribe command topic và khai báo Last Will status

Ý nghĩa: chứng minh P2L Bridge kết nối broker, subscribe nhận lệnh và công bố trạng thái sống/chết của service.

Nguồn: `src/P2L.Mqtt/MqttBridgeService.cs:196-240`

```
private async Task OnConnectedAsync(MqttClientConnectedEventArgs _)
{
    _log.LogInformation("[MQTT] Connected to {Host}:{Port}", _settings.Host, _settings.Po

    // Subscribe to command topic (QoS 1)
    var sub = new MqttClientSubscribeOptionsBuilder()
        .WithTopicFilter(f => f
            .WithTopic(_settings.CommandTopic)
            .WithQualityOfServiceLevel(MqttQualityOfServiceLevel.AtLeastOnce))
        .Build();
    await _client.SubscribeAsync(sub, CancellationToken.None);

    // Publish online status
    _publisher.EnqueueStatus(online: true, gatewayConnected: _gateway.IsConnected);
}

private static MqttClientOptions BuildOptions(MqttSettings s)
{
    // Build LWT payload: broker publishes this automatically on ungraceful disconnect
    var lwtPayload = JsonSerializer.SerializeToUtf8Bytes(
        new BridgeStatusEvt { Type = "bridge_status", Online = false, GatewayConnected =
        P2LMqttJsonContext.Default.BridgeStatusEvt);

    var builder = new MqttClientOptionsBuilder()
        .WithClientId(s.ClientId)
        .WithTcpServer(s.Host, s.Port)
        .WithWillTopic(s.StatusTopic)
        .WithWillPayload(lwtPayload)
```

Nhận xét: Phần code này cho thấy bridge không chỉ “kết nối MQTT”, mà còn subscribe command topic chuẩn và phát `bridge_status` làm Last Will để hệ thống bên ngoài biết khi bridge mất kết nối bất thường.

---

4. Bằng chứng 3 - Chương trình xử lý nhiều loại lệnh MQTT thật

Ý nghĩa: chứng minh hệ thống có command dispatcher thực tế, không phải prototype.

Nguồn: `src/P2L.Mqtt/MqttCommandDispatcher.cs:58–189`

```
switch (type)
{
    case "light_on":
    {
        var cmd = JsonSerializer.Deserialize(payload,
            P2LMqttJsonContext.Default.LightOnCmd);
        if (cmd is null) break;
        var buzMode = cmd.BuzMode ?? cmd.LedMode;
        await _engine.QuickLightAsync(cmd.Address, cmd.Quantity,
            cmd.Color, cmd.LedMode, buzMode, ct);
        break;
    }

    case "batch_start":
    {
        var cmd = JsonSerializer.Deserialize(payload,
            P2LMqttJsonContext.Default.BatchStartCmd);
        if (cmd is null) break;
        var instructions = new List<PickingInstruction>(cmd.Items.Count);
        ...
        await _engine.StartBatchAsync(instructions, ct);
        break;
    }

    case "csv_run":
    {
        var cmd = JsonSerializer.Deserialize(payload,
            P2LMqttJsonContext.Default.CsvRunCmd);
        if (cmd is null || _csvRunner is null) break;
        _csvRunner.Start(cmd.File, cmd.Color);
        break;
    }

    case "scan_devices":
    {
        var cmd = JsonSerializer.Deserialize(payload, P2LMqttJsonContext.Default.ScanDevic
                  ?? new ScanDevicesCmd { Type = "scan_devices" };
        await _scanService.ScanAsync(startAddr: cmd.From, endAddr: cmd.To, ct: ct).Configu
        break;
```

```
    }
}
```

Nhận xét: Đoạn này là bằng chứng trực tiếp rằng chương trình đã có khả năng:

- điều khiển từng module
- khởi động batch picking
- chạy tác vụ CSV
- scan thiết bị trên dải địa chỉ

---

5. Bằng chứng 4 - Business logic batch picking và auto-complete đã có thật

Ý nghĩa: chứng minh chương trình không chỉ gửi lệnh, mà còn quản lý tiến trình picking.

5.1 Khởi động batch

Nguồn: `src/P2L.Core/PickingEngine.cs:51-73`

```
public async Task StartBatchAsync(IReadOnlyList<PickingInstruction> instructions,
                                  CancellationToken ct = default)
{
    if (IsBatchActive)
        throw new InvalidOperationException("A batch is already active. Call FinishBatchAs

    lock (_lock)
    {
        _pendingAddresses.Clear();
        foreach (var inst in instructions)
            _pendingAddresses.Add(inst.Address.PadLeft(4, '0'));
    }

    IsBatchActive = true;

    // Update registry before sending (optimistic)
    foreach (var inst in instructions)
        _registry.SetInstructed(inst);

    await _gateway.InstructBatchAsync(instructions, ct);

    _log.LogInformation("Batch started: {Count} modules", instructions.Count);
}
```

5.2 Tự hoàn tất khi toàn bộ module đã confirm

Nguồn: `src/P2L.Core/PickingEngine.cs:177–193`

```
private void OnConfirmed(ConfirmationEvent evt)
{
    var state = _registry.SetConfirmed(evt);

    bool removed;
    lock (_lock)
        removed = _pendingAddresses.Remove(evt.Address.PadLeft(4, '0'));

    ItemConfirmed?.Invoke(evt, state);

    if (IsBatchActive && removed && _pendingAddresses.Count == 0)
    {
        // Auto-reset so the next batch_start is accepted without requiring
        // an explicit FinishBatchAsync call from the WMS after batch_completed.
        IsBatchActive = false;
        _log.LogInformation("All modules confirmed — batch complete");
        BatchCompleted?.Invoke();
    }
}
```

Nhận xét: Đây là bằng chứng rõ ràng cho thấy batch picking có state thực, có danh sách pending module và có cơ chế phát `BatchCompleted` khi toàn bộ xác nhận hoàn tất.

---

6. Bằng chứng 5 - Giao tiếp phần cứng AIOI có ACK ngay lập tức và ghép response theo sequence

Ý nghĩa: đây là phần quan trọng nhất để chứng minh hệ thống làm việc với phần cứng thật, không chỉ UI hoặc mô phỏng.

6.1 Theo dõi pending response bằng `ConcurrentDictionary`

Nguồn: `src/P2L.Hardware/AioiGateway.cs:28–29`

```
private readonly ConcurrentDictionary<int, TaskCompletionSource<AioiDecoder.ParsedResponse
    _pending = new();
```

6.2 ACK ngay khi thiết bị gửi interrupt

Nguồn: `src/P2L.Hardware/AioiGateway.cs:249–272`

```
switch (response.Kind)
{
    case AioiDecoder.ResponseKind.Interrupt:
        // "t" packet: ACK must echo the same SeqNo as the interrupt (always 0)
        // and must be sent immediately — bypass the send queue so it never waits
        // behind a queued heartbeat or P1 command.
        _ = _transport.SendImmediateAsync(AioiEncoder.Ack(), packet.SequenceNo);

        if (response.Event is not null)
        {
            _log.LogInformation("[CONFIRM] Address={Addr} Status={Status}",
                response.Event.Address, response.Event.Status);
            Confirmed?.Invoke(response.Event);
        }
        break;

    case AioiDecoder.ResponseKind.Ack:
    case AioiDecoder.ResponseKind.Nack:
    case AioiDecoder.ResponseKind.DuplicateSeq:
    case AioiDecoder.ResponseKind.ModuleStatus:
    case AioiDecoder.ResponseKind.Unknown:
        // Route to the pending command completion (matched by SeqNo)
        if (_pending.TryRemove(packet.SequenceNo, out var tcs))
            tcs.TrySetResult(response);
```

Nhận xét: Đoạn code này thể hiện hai bằng chứng quan trọng:

1. khi thiết bị AIOI gửi interrupt xác nhận, chương trình ACK ngay bằng `SendImmediateAsync`
2. các response thông thường được ghép vào lệnh chờ bằng sequence number

Đây là dấu hiệu của một implementation làm việc với thiết bị thật, không phải mock flow.

---

7. Bằng chứng 6 - Web client giữ session sau khi reload và có idle timeout 15 phút

Ý nghĩa: chứng minh phần web client đã có logic vận hành thực tế, không bị disconnect ngay khi người dùng reload trình duyệt.

7.1 Idle timeout 15 phút

Nguồn: `client/app/main.py:49-50`

```
# Idle disconnect: if no WebSocket client is connected for this long, MQTT is stopped.
_IDLE_TIMEOUT_S: int = 15 * 60   # 15 minutes
```

7.2 Bắt đầu đếm giờ khi không còn browser client

Nguồn: `client/app/main.py:211-249`

```
def _on_ws_empty() -> None:
    """All browser clients disconnected — start the idle timer."""
    global _idle_timer
    _cancel_idle_timer()
    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        return
    logger.info("No WebSocket clients — MQTT will disconnect in %ds if idle", _IDLE_TIMEOU
    _idle_timer = loop.call_later(_IDLE_TIMEOUT_S, lambda: asyncio.create_task(_idle_disc

def _on_ws_nonempty() -> None:
    """A browser client reconnected — cancel any pending idle disconnect."""
    if _idle_timer is not None:
        logger.info("WebSocket client reconnected — idle timer cancelled")
        _cancel_idle_timer()

async def _idle_disconnect() -> None:
    """Called after idle timeout with no WebSocket clients — stop MQTT."""
    global _idle_timer
    _idle_timer = None
    if mqtt_client and mqtt_client.is_connected:
        logger.info("Idle timeout (%ds) — disconnecting MQTT", _IDLE_TIMEOUT_S)
        await _do_disconnect()

...
ws_manager.on_empty    = _on_ws_empty
ws_manager.on_nonempty = _on_ws_nonempty
```

Nhận xét: Đây là bằng chứng rất rõ ràng rằng web client đã xử lý bài toán session thực tế: reload trang không làm ngắt ngay, mà chỉ disconnect MQTT sau idle timeout.

---

8. Bằng chứng 7 - Chức năng Scan Devices đã triển khai từ backend tới frontend

Ý nghĩa: chứng minh tính năng scan không chỉ là giao diện, mà đã nối đầy đủ từ API đến UI.

8.1 API backend nhận request scan

Nguồn: `client/app/api/routes.py:419-423`

```
@router.post("/api/scan-devices")
async def scan_devices(request: Request, addr_from: int = 1, addr_to: int = 99):
    if addr_from < 1 or addr_to < addr_from or addr_to > 9999:
        raise HTTPException(status_code=400, detail="Address range must be 1-9999 with fr
    await _publish(request, {"type": "scan_devices", "from": addr_from, "to": addr_to})
```

8.2 Frontend chỉ hiện tab Scan khi MQTT đã connect

Nguồn: `client/static/js/app.js:212-240`

```
function updateConnectionUI() {
  ...
  // Show "Scan Devices" sub-tab only when MQTT is connected
  const scanTabBtn = $('sub-tab-btn-scan');
  if (scanTabBtn) {
    scanTabBtn.style.display = state.mqttConnected ? '' : 'none';
    // If scan sub-tab is active but we just disconnected, switch back to client sub-tab
    if (!state.mqttConnected && scanTabBtn.classList.contains('active')) {
      scanTabBtn.classList.remove('active');
      document.querySelector('#sub-pane-scan')?.classList.remove('active');
```

8.3 Frontend chạy scan, khóa nút và render kết quả trực tiếp

Nguồn: `client/static/js/app.js:632-739`

```
function _renderScanResultsLive() {
  const wrap    = $('scan-results-wrap');
  const summary = $('scan-result-summary');
  if (!wrap) return;
  const entries = [..._scan.found.values()].sort((a, b) => a.address.localeCompare(b.addr
  const count   = entries.length;
  if (summary) summary.textContent = `${count} device${count !== 1 ? 's' : ''} found so fa
  ...
}

async function scanDevices() {
  const btn  = $('btn-scan-devices');
  const from = parseInt($('scan-from')?.value ?? '1',  10) || 1;
  const to   = parseInt($('scan-to')?.value   ?? '99', 10) || 99;

  if (from < 1 || to < from || to > 9999) {
    toast('Invalid address range (1–9999, from ≤ to)', 'error');
    return;
  }

  ...
  if (btn) { btn.disabled = true; btn.textContent = 'Scanning…'; }

  _scan = { fromAddr: from, toAddr: to, found: new Map(), startMs: Date.now(), durationMs

  try {
    await api('POST', `/api/scan-devices?addr_from=${from}&addr_to=${to}`);
  } catch (e) {
    ...
  }
```

Nhận xét: Đây là bằng chứng end-to-end: API scan có thật, UI có điều kiện hiển thị theo trạng thái kết nối, và nút scan bị khóa trong lúc thực hiện để tránh thao tác lặp.

---

9. Kết luận báo cáo

Nhìn từ source code, P2L đã có đủ các lớp quan trọng của một hệ thống triển khai thực tế:

- lớp tích hợp MQTT
- lớp business logic điều phối batch
- lớp giao tiếp phần cứng AIOI
- lớp web client vận hành
- cơ chế scan thiết bị và session persistence

Nói cách khác, đây là một mã nguồn đã đi qua mức ý tưởng / demo giao diện, và đã thể hiện rõ implementation thực tế ở cả backend, MQTT bridge, hardware layer và frontend.